



NAOMS

NAO MEMORY SYSTEM · SOVEREIGN COMPUTE

TRUST TASKS FRAMEWORK · WD 0.6.0

NAOMS & Trust Tasks

A first step toward a long friendship.

We are implementing your framework. This letter introduces who we are, and asks six questions we ran into while building against WD 0.6.0 — **none of them a condition.**

WHAT IS INSIDE

- §01 **What NAOMS is** — one page, including the crates and standards our two stacks already share.
- §02 **Five things we checked before writing** — including the section of yours that already answers half of Proposal 1.
- §04 **Proposal 1 — a party that is itself a set.** The longest section, and the one with production cases behind it.
- §05 **Proposal 2 — the Type URI as a VID.** An internal-consistency argument about your own document.
- §06 **Six questions**, numbered, none of them urgent.

FROM	Mujo · NAOMS sole owner
TO	The Trust Tasks authors — Glenn Gore (editor, Affinidi) and the Trust Tasks Task Force, DTGWG , Trust Over IP Foundation
DATE	2026-09-22
CONTAINS	Two specification changes · eight numbered asks · no conformance claim
SOURCES READ	Trust Tasks framework <code>WD 0.6.0</code> (2026-09-21), three published specifications, and the generated registry index. Full table in §03. Cited throughout by your section number.

00 • Contents

01	What NAOMS is — the stack, where it already meets yours, and where you are ahead
02	Five things we checked before writing — including the section of yours that already answers half of Proposal 1
03	What we read, and what we counted — with the one number we cannot reconcile
04	Proposal 1 — a party that is itself a set
05	Proposal 2 — let the Type URI be a Verifiable Identifier
06	Six questions, and an open invitation
07	Document provenance

01 · What NAOMS is

You have not had a reason to hear of us, so this page exists to give the rest of the document a context.

NAOMS — the NAO Memory System — is a sovereign personal compute and memory system. A person runs their own instance; it holds their data, their credentials and their relationships, and it talks directly to other people's instances. It is pre-release.

◆ The parts, briefly

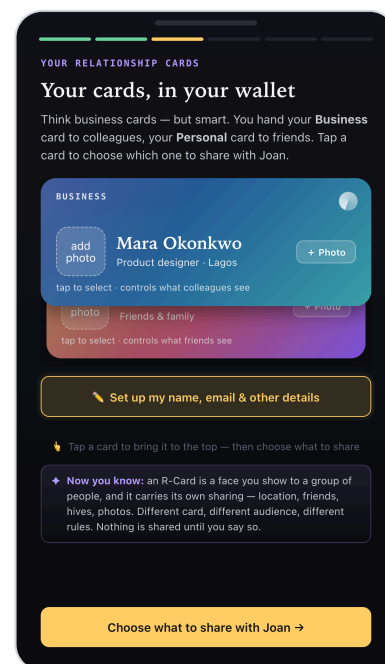
Networking and storage. Peers connect over **Iroh QUIC** — mDNS on a local network, UDP hole-punching between networks, and a relay only as a fallback, carrying ciphertext it cannot read. Storage is local and embedded: **SQLCipher**, encrypted at rest by default, with large objects in a **BLAKE3** content-addressed blob store. **History is a hash-linked DAG in which a commit's identifier IS the BLAKE3 hash of its own canonicalized content, including its parents.** Integrity is therefore structural rather than attested: an entry cannot be restamped or a range quietly dropped without changing every identifier that follows it, so there is no separate log to trust and nothing for an operator to vouch for.

Identity and credentials. `did:key` is the root identity and is what nearly everything uses. `did:naoms` is our protocol-native method, for things reached over the NAOMS protocol rather than the web — a hive, for instance, has no web presence to name, so it is found by asking peers rather than by resolving a host. `did:webvh` appears where a key must rotate with a verifiable history. Signing keys live in a separate Rust signer process, and **FROST** threshold signatures let authority be held by a group rather than by a device. What a person sees is an **R-card** — a relationship card — and underneath these are W3C-shaped verifiable credentials, with **BBS+** selective disclosure and **W3C BitstringStatusList** revocation.

Messaging. **DIDComm v2** on the live inbound and outbound paths, for credential exchange and group-key transport; **MLS (RFC 9420)** for media and calls.

◆ Where our work already meets yours

We went and measured this rather than asserting it, because it is the part of the letter we most wanted to be true and therefore the part we least trusted ourselves on. **Three dependencies are the same crate at the same version in both projects:** `bls12_381_plus 0.8.18` under our BBS+ (you reach it through



Onboarding: choosing an R-card. Relationship cards are the unit a person actually sees. The names shown are fictional.

`affinidi-bbs`, we through `zkryptium`), `bip39 2.2.2`, and `subtle 2.6.1`. Several more are the same crate at a different major or minor: `ed25519-dalek`, `x25519-dalek`, `hkdf`, `hmac`, `chacha20poly1305`, `argon2`.

At the standards layer the overlap is larger: **DIDComm v2**, **RFC 8785 JCS** canonicalization, **did:webvh** (ours is used for key rotation and is not network-published, so this is a shared method rather than a shared deployment), **W3C BitstringStatusList**, **Argon2id**, **MLS RFC 9420**. **EVIDENCE** And there is a lineage we did not expect to find: the DIDComm library we use, `didcomm@0.4.1`, is a WebAssembly build of `sicpa-dlab/didcomm-rust`, and your design notes record testing against the Sicpa reference vectors. **Two projects that never spoke are both anchored to the same reference implementation.**

◆ What we do differently, stated as difference

The architectures diverge most at the network and at the trust path. **Your stack reaches a party through infrastructure** — DNS names, an HTTPS registry, an identity server, a DIDComm mediator that is also the NAT answer — and the security model is built to make those components untrusted, which it does well: the mediator carries end-to-end authcrypted payloads it cannot read. **Ours reaches a party directly** — on a local network there is no intermediary at all, and that is not a configuration setting but what the transport does. We have no origin host to name and no certificate to present.

ASSESSMENT **We do not think one of these is the correct answer and the other a mistake.** They are different approaches, chosen against different constraints: an infrastructure path is operable, auditable and deployable in ways a peer-to-peer path is not, and a great deal of what you have designed is harder precisely because you did not give yourselves the shortcuts we took. The reason we state the difference at all is narrow and practical: it is why we cannot emit a Type URI that conforms to your **§6.1**, and it is the whole of Proposal 2. Nothing else in this letter depends on it.

◆ Why we are adopting Trust Tasks

NAOMS runs whole: no origin server, and nothing in the list above needs a second party to function. We describe it that way only because it explains what we recognised in your framework — **we had arrived at the same idea from the other side, and had not written it down.**

What WD 0.6.0 has that we did not is the writing-down. We had already concluded that an operation must declare its own consequence, and we put that declaration in our manifest beside the handler, where it cannot drift from the code it describes. What we skipped is the part you did: a self-contained document with a proof over it, a Type URI that names a version, a stability policy that says what may break and when, and a conformance section that lets someone else check an implementation without asking us.

ASSESSMENT **Those are the parts that make a private discipline into something two parties can share, and they are harder to get right than the idea they carry.** Reading §7.2 item 10 — that a validated identity is not an authorization — and §9.1.1’s “silence is not permission” is what persuaded us: those are rules written by people who had already been bitten, and we would rather inherit them than re-earn them.

It has already changed our code. Our credential proofs previously signed a canonicalized body under a proof type name of our own, which did not interoperate with your verifier. **They are now W3C Data Integrity proofs under** `eddsa-jcs-2022`, and this reading of your framework is the reason we moved.

ASSESSMENT **There may be something in this for you as well.** §5.3 makes *two independent, interoperable implementations* the entry criterion for `candidate`. We are a project with a different codebase, a different language, a different transport and no relationship to yours — which is close to the definition of the independent second implementation that criterion is asking for. **If our adoption helps WD 0.6.0 move off Working Draft, that is a better outcome for us than anything we could ask you for.**

Claims about our own stack are measured from our `Cargo.lock` and source, and claims about yours from your published `Cargo.toml`, registry index and design notes; where both lock files name the same crate we compared exact versions, not crate names. One caveat we would rather state than have found: our canonicalization sorts keys and serialises with JavaScript's `JSON.stringify`, which coincides with RFC 8785 for our value domain but is not an independent implementation of the RFC.

02 · Five things we checked before writing

This document proposes two changes to a specification we did not write. Before any of that, here is what we checked about our own position — including the two places where the answer went against us. None of it should have to be extracted from us later.

◆ 1 — Where we are in the implementation

EVIDENCE We are implementing Trust Tasks, and everything in this letter came out of doing that.

We read WD 0.6.0 closely, mapped it onto a system that already performs the work it describes, and began building against it — and the two proposals below are the places where the mapping did not close. **We have not yet certified ourselves against §7.1 or §7.2**, so please do not read this as a conformance report; read it as notes from the workbench. That is the whole basis of our standing to write, and we would rather you weighed the arguments knowing it than discovered it afterwards.

◆ 2 — The bilateral document model is a decision, and for most tasks it is the right one

§3 (*Terminology*, the *Trust Task* entry) states it without ambiguity:

The document model defined by this framework is bilateral: each Trust Task document names at most one issuer and one recipient. Exchanges involving more than two parties are modeled as multiple bilateral Trust Tasks linked by the framework's `threadId` member.

We are not calling this a gap you overlooked. It is a considered decision with real payoffs — one object, one schema composition, and an identity rule (§4.8.1) that has exactly two sides to compare rather than n . For a large class of multi-party work it is also simply *true* to the thing being modelled. **A vote genuinely is each member receiving their own request and answering individually**; one document per member, linked by a thread, is the honest encoding of that, and we would not argue against it. Proposal 1 is not an argument that the decomposition is wrong. It is an argument that it is not *total*.

◆ 3 — §4.8.3 already answers half of what we were going to ask for

This is the concession that cost us the most, and we are putting it in bold because an earlier draft of this document did not contain it. We were going to present bearer-style, audience-free documents as something the framework has no answer for. **That would have been false.** §4.8.3 (*Bearer Specifications*) defines exactly such an opt-out: a specification may declare `bearer: true`, declare `recipient` **OPTIONAL**, and state in prose why audience binding is inappropriate — and §7.2 item 8 and §7.3 item 12 carry the mechanism through to the consumer and to the front matter. Any argument of ours that ignores §4.8.3 is an argument that has not read you.

ASSESSMENT What survives the concession is narrower, and we state it narrowly in §04: **§4.8.3 removes the audience; it does not let you name one that is a set.** Those are different things, and the second is what a bearer capability minted for a bounded-but-unenumerated group actually needs. §4.8.3 is explicit that it is the first: a bearer specification makes a normative claim that its documents are intended for *unspecified* consumption, **SHOULD NOT** carry any payload field whose interpretation depends on the receiving party's identity, and **MUST NOT** be declared bearer unless the audience-free property is *intrinsic* to the assertion. A capability that is redeemable by any member of a specific group, and by nobody else, fails all three tests while still having no enumerable recipient list at emit time.

◆ 4 — A count of ours did not survive re-measurement, and we are printing the measured one

We came into this holding a figure of **483** distinct task slugs in your catalogue, with `vtc/*` at 90 and `vta/*` at 80. **Re-measured against the registry index we actually hold, none of those three numbers reproduces.** We get 407, 79 and 72. The full census and the exact instrument are in §03, and we have **not** silently substituted the new numbers for the old and moved on — question 6 in §06 asks which denominator the project uses, because the gap is large enough that we think we are counting a different thing rather than counting wrong.

◆ 5 — We have an interest in Proposal 2, and we are declaring it rather than burying it

Proposal 2 asks you to relax a rule in §6.1 that protects something real. **NAOMS uses no domain names anywhere** — not for identity, not for discovery, not for code distribution — because DNS is a centralisation we removed by design. **So we are the party that benefits if the rule changes, and as §6.1 and §6.5 stand today we cannot emit a conforming *Type URI* at all.** That is a motive, and a reader is entitled to discount an argument for knowing it. We think the argument stands on its own — it is about the internal consistency of your document, not about us — but you should not have to work out our interest for yourselves.

Four of these five concessions are about our own position rather than yours. The fifth, §4.8.3, is a correction to a draft of this document that you never saw; it is included because the shape of an argument we *nearly* sent you is information about how carefully to read the one we did.

03 • What we read, and what we counted

Every claim in this document about your specification is cited by **your section number**, the way we would cite our own code by `file:line`, so that each is checkable by you without reference to anything of ours. This section fixes what “your section number” resolves against.

◆ The documents this was read at

ARTIFACT	REVISION AS READ
Trust Tasks framework specification	Document Status: Working Draft 0.6.0 ; Version: 1.0 (not yet ratified, per §5.1.1); <i>Last Updated</i> 2026-09-21; generated from 8c63241a65c0 of dtgwg-trust-tasks-spec
acl/change-role 0.1	status: draft , targetFrameworkVersion: 0.5.0
task-consent/request 0.1	status: draft , targetFrameworkVersion: 0.5.0
vetting/session 0.1	status: draft , targetFrameworkVersion: 0.5.0
Generated registry index	Header states <i>generated at</i> 2026-09-22T07:23:02.032Z by scripts/build-registry.mjs

Section numbers throughout are the ones carried by the generated framework document, which notes that they are assigned in canonical order because the registry cites the framework by number, and are not part of the canonical text. Where that mapping has moved since 2026-09-21, the quoted sentence rather than the number is the thing we mean.

◆ The census, and the number we cannot reconcile

EVIDENCE Counted by loading the generated registry index and counting its entries directly, rather than by pattern-matching text:

QUANTITY	MEASURED	WE HAD EXPECTED
Entries in the index (slug × version rows)	471	—
Distinct slugs	407	483
vtc/* — distinct slugs / rows	79 / 88	90
vta/* — distinct slugs / rows	72 / 86	80
Rows at status: draft	418	—
Rows at status: retired	53	—
Rows at candidate or standard	0	—
Distinct categories	16	—

We are reporting this gap rather than resolving it, deliberately. Neither of our two defensible denominators — 407 distinct slugs, or 471 slug-version rows — is 483, and the three discrepancies do not share a constant offset, so this is not one snapshot being newer than another. **The likeliest explanation is that we are counting a different object than the project counts** — that the catalogue figure includes bindings, ceremonies, shared schema components, or response variants that the generated index does not carry as rows. **That is question 6 in §06.** We would rather publish a number of yours that we understand than a number of ours that happens to be ours.

◆ One observation about maturity, stated as measurement and not as criticism

EVIDENCE No specification in the index has reached candidate or standard. 418 rows are draft; 53 are retired, which §5.3.1 item 3 makes reachable directly from draft (“abandoning a working draft”) and which §5.3 makes terminal. The framework document itself is Working Draft 0.6.0.

ASSESSMENT For a registry of this size at this age, that is unremarkable and we do not present it as a finding against the work. §5.3 sets a high bar for candidate — *two independent, interoperable implementations* — and a bar that is doing its job produces exactly this distribution early on. We raise it only because it is the input to a planning question (ask 6), and because your own document already names what is outstanding: §15 and §16 each state that their guidance “will be completed before this specification advances beyond Working Draft status,” and §17.2 states that a conformance test suite “has not yet been published” and that a normative one “will be defined” before the same threshold. **Those are your exit criteria, not ours.**

◆ How to read the two labels in this document

- **EVIDENCE** — a quotation from your text, or a count we produced with an instrument described well enough for you to rerun it. If it is wrong, it is wrong in a way you can demonstrate.
- **ASSESSMENT** — our judgement about what a measurement means or what ought to follow from it.

Every proposal in this document is an assessment. Disagreeing with one costs you nothing in credibility with us.

Where a claim below carries neither label it is a statement about NAOMS, which you cannot check and which we therefore keep to the minimum the argument needs.

04 · Proposal 1 — a party that is itself a set

The ask is not “let `recipient` be a list.” It is that the framework gain a way to name a party that is *itself* a set, because `threadId` decomposition silently assumes that every multi-party exchange is a collection of individual bilateral ones. That assumption is true for a vote. It is false for a threshold, and false for a bearer audience.

◆ What the decomposition assumes

§3 models an n -party exchange as n bilateral tasks sharing a `threadId`, and §4.9 gives that member its semantics: it correlates, it carries no normative validation semantics, and §4.9 requires consumers not to reject a document on the basis of `threadId` alone. **ASSESSMENT** **That is a correlation mechanism, and correlation is exactly the right primitive when the underlying reality is n separate acts that happen to belong together.** The decomposition is lossless precisely when each bilateral task is independently meaningful — when member k ’s answer is a complete act whether or not members 1 through n ever answer.

Our two cases are the two ways that condition fails.

◆ Case 1 — authority held by the group rather than by its members

NAOMS holds certain authorities under a **t -of- n threshold signature**. The distinguishing property is not that several parties are involved. It is that **a threshold signature is not n individual acts; it is one act that requires n participants**. No single share is a signature, no share is independently meaningful, and the authority being exercised does not reside in any member — it resides in the group and is inaccessible to every proper subset smaller than t .

ASSESSMENT **There is therefore no member who could honestly be named as the `recipient` of a request to exercise that authority.** Naming one asserts something false: that this party is the one the issuer expects to act upon the document (§4.8). It is not — it cannot act on it alone, and whether the act occurs at all depends on parties the document does not name. Decomposing into n bilateral requests produces n documents each of which is individually unsatisfiable, and the one thing that *is* true — that the group holds the authority — is the thing no document states.

Where this bites in the framework, precisely

The obvious answer is to name the group in the `payload`, where a specification may define whatever structure it likes (§4.5, and Design Goal 3 in §1.1). **That works for description and fails for everything else**, and §7.3 item 5 says so in your own words:

A specification identifies which party fills each framework member by tagging that party `issuer` or `recipient`; a party named only in the payload — neither the document's `issuer` nor its `recipient` — carries no such tag.

EVIDENCE A party in the payload is outside every identity rule the framework defines. It is not reached by the in-band-over-transport precedence of §4.8.1; not reached by §7.2 item 5, which is what makes a `recipient` enforceable at all; not reached by §7.2 item 6, the transport cross-check; and not covered by the audience binding of §4.8.2, which §7.2 item 8 turns into a hard `malformedRequest` rejection. **The framework's entire identity apparatus attaches to two members, and a group-held authority can occupy neither.**

The bind tightens rather than loosens when the document matters most. §4.7.1 makes `proof` **MUST** where a strong, transport-independent guarantee is required — which is the normal case for anything a threshold signature guards. §4.8.2 then requires an in-band `recipient` whenever `proof` is present and the specification is not a bearer one. **So the documents that most need to name the group are exactly the documents the framework obliges to name a single VID instead.**

◆ Case 2 — audiences that are not enumerable when the document is emitted

NAOMS mints **bearer capabilities whose redeemer set has no cardinality at the moment the document is created**. The capability is issued; who will present it, and how many will, is not known and is not knowable at emit time. You cannot decompose into one-document-per-recipient when the recipients are not yet determined.

§4.8.3 is the right place to look, and we said so in §02 before making this argument. A bearer specification opts out of §4.8.2 entirely: `bearer: true`, `recipient` declared **OPTIONAL**, and prose stating why audience binding is inappropriate. If the only requirement were “no recipient at emit time”, §4.8.3 would close this case and Proposal 1 would be a one-case proposal.

ASSESSMENT The reason it does not close is that §4.8.3 gives you “no audience”, and what a capability needs is “an audience that is a set”. §4.8.3 requires the audience-free property to be *intrinsic* to the assertion — its examples are a public attestation, a heartbeat, a schema-publication announcement, documents that are true toward the whole world — and it adds that a bearer specification **SHOULD NOT** carry any payload field whose interpretation depends on the receiving party's identity. A capability redeemable by *any member of this group and by nobody else* violates all three: its audience is bounded, the bound is the substance of the grant rather than an accident of delivery, and whether the holder may redeem it depends entirely on who the holder turns out to be.

Declaring such a specification `bearer: true` to get past §4.8.2 would be conformant to the letter and dishonest in substance — it publishes a normative claim that the document is intended for unspecified

consumption, which is the opposite of what a bounded capability asserts. **We would rather ask for the missing expressiveness than take the opt-out under false pretences.**

◆ What we are actually asking for

A way to name, in the framework's own identity members, a party whose identity is *set-valued* — along with whatever predicate makes membership checkable by the consumer. We are deliberately not proposing the encoding, because the encoding is yours and because more than one would work. What matters is the property:

- 1. The named party is a set, and the framework knows it is.** A consumer can tell a set-valued party from a single VID without reading the payload schema, and the §4.8.1 precedence, §7.2 item 5 enforcement, and §4.8.2 audience binding all have a defined meaning for it.
- 2. Membership is checkable, and the check is the consumer's.** Exactly as §7.2 item 10 already insists for everything else: establishing that a party is in the set establishes *who*, never *authorized*.
- 3. The set need not be enumerated at emit time.** A threshold group is bounded and known; a capability audience is bounded and not yet known. A predicate covers both; a list covers only the first.

What we are not asking for. Not that the bilateral default be dropped — it should stay the default, and most specifications should keep using it. Not that the framework adopt a threshold signature scheme, define a group-key format, or take any position on how a set proves its own membership; that is cryptography and it belongs in a Trust Task specification or a transport binding, not in the envelope. Not that `recipient` change type. **The whole of the ask is that the framework have a place to say “this party is a set”, so that the two cases above stop being inexpressible rather than merely inconvenient.**

◆ A drafting note, which is not the argument

EVIDENCE The Abstract opens “a specification for the verifiable work that occurs **between two or more parties**” and §1.1 frames the design goals around problems arising “wherever **two or more parties** cooperate over a network”, while §3 defines a *Trust Task* as “a unit of verifiable work **between two parties**” and states the bilateral model outright.

ASSESSMENT **This is a drafting inconsistency, not evidence for Proposal 1, and we would be misrepresenting it if we led with it.** Both readings are defensible — §3 describes the *document* and the Abstract describes the *framework*, which does address *n*-party exchanges by decomposition. We mention it only because a reader arriving at §3 from the Abstract may believe the model is *n*-ary and discover otherwise several sections later, and one clause in the Abstract would remove that.

Both production cases above are NAOMS-side facts that you cannot verify from outside, and we have kept them to the minimum the argument requires. The framework claims in this section — §3, §4.5, §4.7.1, §4.8, §4.8.1, §4.8.2, §4.8.3, §4.9, §7.2 items 5, 6, 8 and 10, and §7.3 item 5 — are all checkable in your own text.

05 · Proposal 2 — let the Type URI be a Verifiable Identifier

§6.1 requires the scheme of every *Type URI* to be `https`, and gives the reason in the same sentence:

*The URI scheme **MUST** be `https`. Other schemes (including `http`) are non-conformant: every representation served at a Type URI depends on transport-layer authentication and integrity, and permitting `http` would normalize a transport-downgrade path for any consumer that dereferences the URI.*

We are not asking you to weaken that requirement. The downgrade concern is real and the rule as written defends against it correctly.

◆ The rule protects a property; it is written as a mechanism

ASSESSMENT What the requirement actually protects is a **property**: *a consumer can trust that what it resolved is what the identifier names*. `https` is one **mechanism** for obtaining that property, and it obtains it by trusting two third parties the specification never names — the DNS hierarchy that resolves the authority, and a certificate authority that vouches for it. A self-certifying identifier delivers the same property cryptographically, trusting neither: the identifier *is* a commitment to the key that authenticates the content, so verification is a local computation rather than an appeal to an external namespace.

Stated as a property, the rule would cover both — and would continue to exclude `http`, which supplies the property by no mechanism at all. **That is the entire change we are proposing: state the requirement as the property you want, rather than as the single mechanism you had in mind.**

◆ The sharper point: your own document already does this for parties

§3 defines every party as identified by a Verifiable Identifier, and §4.8 makes both `issuer` and `recipient` VIDs. §3 is explicit that the framework *does not constrain the VID scheme* — DIDs, X.509 subjects, OIDC subject identifiers, key thumbprints — and that the consumer's trust framework decides which are accepted and how each resolves to verification material. **So the framework already relies on self-certifying identifiers for the actors, and then identifies the specifications themselves by a DNS name.**

ASSESSMENT That is internally inconsistent, and it is notably so in a **Trust Over IP** deliverable, whose premise is not depending on DNS for identity. The asymmetry is hard to motivate from first principles: a party identifier and a specification identifier are both long-lived names that a consumer must resolve to authentic material, and if anything the specification identifier is the more durable of the two — §6.4 requires the representations at a `candidate`, `standard` or `retired` Type URI never to change their normative content, a commitment measured in years, resting on a domain registration measured in one-year renewals.

ASSESSMENT **The cryptography gives no reason for the asymmetry either.** The ecosystem's Data Integrity profile is JSON-native: `task-consent/request/0.1` notes, non-normatively, that “the reference ecosystem signs this document with the `eddsa-jcs-2022` Data Integrity cryptosuite and `proofPurpose: assertionMethod`”, and the examples in `vetting/session/0.1` carry the same suite. A JCS-canonicalized, Ed25519-signed proof touches DNS nowhere. **The `https`-only Type URI requirement is therefore an orthogonal constraint inherited from how the registry happens to be hosted, not something the document's own cryptography needs.**

◆ The change is smaller than it sounds

DID URLs carry paths and query strings. Your `<slug>` and `<MAJOR.MINOR>` structure therefore survives unchanged — and so does the resolution rule you wrote for it in §6.1, which identifies “the version as the final path segment... and the slug as the segments between `/spec/` and the version.” That rule is written against path structure and is indifferent to the authority:

```
https://trusttasks.org/spec/acl/grant/0.1    ← today, unchanged
did:example:registry/spec/acl/grant/0.1    ← same slug, same version,
                                           same parse
```

Nothing else in the specification moves. The slug grammar (§6.1), the reservation rule on `trust-task*` and `trust-ceremony*` (§6.1, and §6.5 item 2 which applies it regardless of authority), the `#request` / `#response` fragments (§4.4.1), the structural disjointness of `/spec/` from `/binding/` and `/ceremony/` (§6.1, §9.3, §6.7), and the version grammars of §5.1 and §5.1.1 all operate on path and fragment structure alone.

The one place it genuinely touches, stated against ourselves

ASSESSMENT **§6.2 is the real dependency, and we will not pretend otherwise.** Content negotiation over `Accept` is an HTTP mechanism, and “a server hosting a *Type URI* **MUST** support HTTP content negotiation” has no literal reading for a VID. Any VID-form Type URI needs an answer for how the three representations of §6.2 — `text/html`, `application/schema+json`, `application/ld+json` — are obtained and authenticated.

We think the framework already contains the shape of that answer rather than needing a new one. **§6.5 item 4 contemplates a Type URI that is unresolvable and a consumer that “relies on out-of-band distribution of the specification document and schema”**, and §7.2 items 1 and 2 require validation

against the schema for a declared version without requiring that the schema was fetched at that instant over that scheme. **Content negotiation is how the registry serves representations; it is not what makes a Type URI an identifier.** Separating those two would let §6.1 name the property, §6.2 keep describing the HTTPS registry exactly as it does today, and a VID-form Type URI satisfy the first while meeting the second by a different route.

◆ Our own position, stated plainly

NAOMS does not use domain names anywhere. They are a centralisation we removed by design — there is no origin host to name, and no certificate to present. Under §6.1 as written we therefore cannot emit a conforming Type URI; and §6.5, which exists precisely for specifications outside the public registry, does not help us, because its item 1 still requires the URI shape of §6.1 and the `https` scheme requirement applies to both the public and the private form. **A private specification of ours would be non-conformant on the scheme alone, with everything else in §7.3 satisfied.**

ASSESSMENT We are not asking to be accommodated. We are pointing out that the framework currently excludes a class of implementer it otherwise fits well — one that has adopted the self-certifying identity model the rest of the document is built on — and that the exclusion is a side effect of a rule aimed at something else entirely.

Framework claims in this section: §3 (*Party*, *VID*), §4.4.1, §4.8, §5.1, §5.1.1, §6.1, §6.2, §6.4, §6.5 items 1, 2 and 4, §6.7, §7.2 items 1 and 2, §7.3, §9.3. The cryptosuite observation is from the non-normative note in `task-consent/request/0.1` and the examples in `vetting/session/0.1`.

06 • Six questions, and an open invitation

Six questions. The first two are the proposals; the rest are things we could not answer from the published material and that shape what a downstream implementer can plan against. **None of them is urgent, none of them is a condition, and a “no” to any of them is a useful answer.**

◆ The two proposals

1. **Rule on a set-valued party (§04).** Is there a way for the framework to name a party that is itself a set — with a consumer-checkable membership predicate, not an enumerated list — so that group-held authority and bounded-but-unenumerated audiences become expressible? **FRAMEWORK CHANGE** We would be glad to be told that this belongs in a transport binding or in an individual specification rather than in the envelope; that is a legitimate answer and it would settle where we should be looking. The one answer we would struggle with is the `threadId` decomposition, because for these two cases it does not record what actually happened — but if that is the answer, we would rather understand why than argue with it.
2. **State §6.1’s scheme rule as a property rather than a mechanism (§05), and allow a *Type URI* to be a VID on the same terms §3 already allows for a party.** **FRAMEWORK CHANGE** If the answer is no, the most useful form of the no is *which* property of `https` a self-certifying identifier fails to supply — that would tell us whether the obstacle is technical or is about registry operations, and those need different work from us.

◆ Two drafting observations, offered without a position

3. **EVIDENCE** Is the `eddsa-rdfc-2022` proof in the `acl/change-role/0.1` example deliberate? That specification’s example document carries `"cryptosuite": "eddsa-rdfc-2022"`, while `task-consent/request/0.1` notes that the reference ecosystem signs with `eddsa-jcs-2022` and the `vetting/session/0.1` examples use that suite throughout. **§4.7 leaves the choice open, so both examples are conformant and neither is wrong.** We raise it only because the examples are what implementers copy, and the two suites canonicalize differently enough that copying the wrong one is an expensive afternoon.
4. **EVIDENCE** Is a mandatory-to-implement cryptosuite intended, and if not, how should two implementations expect to verify each other? §4.7 leaves the cryptosuite choice open and names no floor, and the published material exercises that openness in both directions — `task-consent/request/0.1` notes the reference ecosystem signing with `eddsa-jcs-2022`, while the `acl/change-role/0.1` example carries `eddsa-rdfc-2022`. **Both are conformant, which is exactly what makes this worth asking:** an implementation that supports only one is also

conformant, so two conforming implementations are not guaranteed to verify each other's documents. The two suites canonicalize by different routes — JCS over the JSON, versus RDF dataset canonicalization — and the second is a substantially larger dependency to take on.

ASSESSMENT We raise it because §5.3 makes *two independent, interoperable implementations* the entry criterion for `candidate`, and a floor is the cheapest thing that makes that criterion reachable. Naming one suite as mandatory-to-implement, while leaving others permitted, would settle it without narrowing the framework.

◆ Two planning questions

- 5. Do you expect the specification to stabilise, and on roughly what horizon?** **ASSESSMENT** **This is a planning question and not a criticism.** We observe that no specification in the registry index has reached `candidate` or `standard` (§03), and your own document names three things outstanding before it leaves Working Draft: internationalization guidance (§15), accessibility guidance (§16), and a normative conformance test suite (§17.2, which states that none has yet been published). **What turns on the answer is whether a downstream implementer can pin.** §6.4 offers a firm stability commitment at `candidate` and above and deliberately offers none at `draft`, and §5.2 lets a `draft` make a breaking change under a `MINOR` increment — which is the correct design for a draft and also means that today there is nothing for an implementer to pin to. A rough answer, even “not before the test suite exists,” is worth more to us than a precise one.
- 6. Which denominator does the project use for the catalogue?** We measure **407 distinct slugs** across **471** slug-version rows in the generated index, against a figure of 483 we were carrying; the per-prefix counts disagree too (`vtc/*` 79 slugs / 88 rows, `vta/*` 72 / 86). **We have printed our measurement and flagged the gap rather than picking whichever number suited the sentence** (§03). If the published figure counts bindings, ceremonies, shared schema components or response variants alongside specifications, we would like to count the same object you do.

◆ And the standing invitation

Correct us. Every framework claim in this document is cited to a section of yours so that each one can be checked without reference to anything of ours, and §02 lists the five things we checked before arguing — one of which, §4.8.3, is a correction we made to ourselves late enough that an earlier draft would have misrepresented your work to you. **If any citation here is wrong, or any inference from a correct citation is wrong, that is the most useful reply this document could receive**, and it would be more welcome than agreement.

ASSESSMENT The honest summary of our position: **we read WD 0.6.0 as a careful document that has thought harder about several things than we had** — §7.2 item 10's refusal to let identity validation stand in for authorization, and §9.1.1's “silence is not permission,” are both rules we wish we had written down earlier ourselves. Our two proposals are about expressiveness at the edges, not about the design.

Neither of them is a reason not to build against this framework, and we are not treating them as one.

07 • Document provenance

This document was prepared by NAOMS and is addressed to the authors of the Trust Tasks framework specification. It was written against **Trust Tasks framework** **Working Draft 0.6.0**, *Last Updated* 2026-09-21, generated from commit `8c63241a65c0` of `dtgwg-trust-tasks-spec`; against the published specifications `acl/change-role/0.1`, `task-consent/request/0.1` and `vetting/session/0.1`; and against the generated registry index whose header states it was produced at **2026-09-22T07:23:02.032Z**. Every claim about your work is cited to a section number of yours, and the counts in §03 were produced by loading the registry index and counting its entries rather than by matching text, so that both the figure and the instrument are yours to re-run. Where a measurement of ours disagreed with a figure we were carrying, **the disagreement is printed rather than resolved** (§03, and question 6). Where a claim is a judgement rather than a measurement it carries an **ASSESSMENT** chip, and nothing in this document depends on a source we have not read. Corrections to *this* document are as welcome as the proposals it makes.

About NAOMS. NAOMS — the NAO Memory System — is a sovereign personal compute and memory system: peer-to-peer, with no origin server, no domain names, and no third party in the trust path. Its design rests on **three axioms**, which no policy, procedure or convenience may override.

- **Wholeness** — the system is complete in itself; no external dependency is required for any core function. It is the axiom behind §05: a name that must be resolved through DNS and a certificate authority is a core function held by someone else.
- **Honesty** — cryptographic and epistemic. No silent mutation, no silent drop, no silent clamp; and no claim asserted beyond its evidence. It is why §02 precedes §04, why §03 prints a count that contradicts our own, and why §04 declines a conformant route through §4.8.3 that would have stated something untrue.
- **Mystery** — epistemic humility; forgetting is a feature, and so is saying what one does not know. It is why the asks in §06 are questions rather than positions.

Prepared by Mujo, sole owner of NAOMS, 2026-09-22. NAOMS is not a member of the Trust Over IP Foundation and this document is not a submission under any ToIP process; it is one implementer writing to another about a specification it read carefully. The Trust Tasks framework specification is published under CC BY 4.0 by the Decentralized Trust Graph Working Group; the quotations above are reproduced under that licence for the purpose of citing them precisely.